

# Module 27: Composition, Aggregation & Association

*Introduction to C++ Programming — Part 3: Object-Oriented Programming with C++*

---

## Module Overview

---

Module 11 introduced inheritance - 'is-a' - and beginners overuse it. Most relationships between classes are some form of 'has-a' or 'knows-about'. This module gives students the vocabulary and the C++ mapping for each: composition (parts that live and die with the whole, held by value or `unique_ptr`), aggregation (parts that exist independently, held by non-owning pointers or `weak_ptr`), association (peers that know about each other, including keeping two-way links consistent), and dependency (temporary use through a parameter). It adds the remaining UML relationship symbols and makes the case for composition over inheritance, including the 'class explosion' problem and delegation.

**Suggested Duration:** 75–90 minutes

## Learning Objectives

---

By the end of this module, students will be able to:

- Distinguish is-a from has-a, and reject inheritance where is-a sounds wrong.
- Identify composition, aggregation, association and dependency, and choose between them by asking about lifetime.
- Implement each relationship with the matching C++ technique (value member, `unique_ptr`, raw pointer/reference, `weak_ptr`, parameter).
- Keep a two-way association consistent by changing it through a single method.
- Draw relationships in text UML with the correct symbols and multiplicities.
- Explain and apply 'prefer composition over inheritance', using delegation.

## Prerequisites

---

- Module 26 (smart pointers and ownership vocabulary).
- Module 20 (UML class boxes).

## Materials & Instructor Prep

---

- 27-composition-aggregation/examples/ 01 to 04; 04\_composition\_over\_inheritance.cpp is the persuasive one.
- A large UML relationship-symbol chart on the board for the whole session.

## Key Vocabulary

| Term         | Definition                                                                                                      |
|--------------|-----------------------------------------------------------------------------------------------------------------|
| Composition  | A strong has-a: the whole owns its parts, which are created and destroyed with it and belong to one whole only. |
| Aggregation  | A weak has-a: the whole groups parts that exist independently and may belong to several wholes.                 |
| Association  | A longer-term 'knows-about' relationship between peers, one-way or two-way.                                     |
| Dependency   | Temporary use of another class, typically as a parameter or local variable, with nothing stored afterwards.     |
| Multiplicity | How many objects take part at each end of a relationship: 1, 0..1, *, 1..*.                                     |
| Delegation   | A whole forwarding a request to one of its parts, choosing exactly which of the part's abilities to expose.     |

## Lesson Outline & Instructor Talking Points

### 1. is-a vs has-a (10 min)

- Read pairs aloud and have the class vote is-a or has-a: TextBook/Book, Car/Engine, Robot/Motor, SmartPlug/Device.
- State the rule: if 'is-a' sounds wrong, inheritance is wrong, even if it would reuse code.

### 2. Composition and aggregation (20 min)

- Run examples/01\_composition.cpp: the robot's parts are built before it and destroyed after it, with no pointers or destructor.
- Run examples/02\_aggregation.cpp: closing the departments leaves the teachers alive, and one teacher is in two departments.
- Write the lifetime test on the board: 'If I destroy the whole, should the part be destroyed too?'

**Instructor note:** Map every relationship back to Module 26's vocabulary: composition owns; aggregation and association borrow or observe. That link is what makes the abstract categories concrete.

### 3. Association and dependency (15 min)

- Walk through examples/03\_association.cpp. Ask what would go wrong if outside code set the doctor's list and the patient's pointer separately.
- Show a dependency as a parameter-only relationship, and suggest preferring it to a stored member when something is only needed in one method.

#### 4. UML relationships (10 min)

- Draw each symbol with an example: filled diamond, hollow diamond, plain line, dashed arrow, hollow triangle; add multiplicities.
- Have pairs draw a five-class school diagram with at least one of each relationship.

#### 5. Composition over inheritance (20 min)

- Run examples/04\_composition\_over\_inheritance.cpp. BadSensorLog accepts 9999 C and can be cleared by anyone; SensorLog cannot.
- Draw the class-explosion tree for robots with optional parts, then show the composed Robot whose parts can even change at runtime.

**Instructor note:** *This is the most transferable idea in the module. Many professional codebases have suffered from deep inheritance trees; students who absorb this early design better.*

### Common Student Misconceptions

| Misconception                                                                         | How to Address It                                                                                                                                              |
|---------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| "Inheritance is the main way to reuse code."                                          | Composition plus delegation reuses code while exposing only what you choose. Inheritance is for genuine is-a relationships that need polymorphism.             |
| "Holding a raw pointer to a part is composition."                                     | If the whole owns the part, ownership should be visible and automatic: a value member or a <code>unique_ptr</code> . A raw pointer signals a borrow.           |
| "Aggregation and association are the same in C++, so the distinction doesn't matter." | The code may look similar, but the meaning - whole/part vs peers - guides design decisions such as who may change the link and what multiplicities make sense. |
| "Inheriting from <code>std::vector</code> is a quick way to make a custom list."      | It exposes every vector operation and makes the class's rules impossible to enforce. Hold a vector as a member instead.                                        |

### Exercises & Module Project

Exercise: 27-composition-aggregation/exercises/exercise1.cpp - model ICT-lab computers: Processor and Memory by value, polymorphic Storage in a `unique_ptr`, shared Peripherals by non-owning pointer, a TechnicianReport dependency, and a UML diagram. Project (three tracks): Generic - school structure; EE - smart-home wiring; Biomedical - hospital structure. Each writes DESIGN.md with UML and a relationship table, splits the model into one file per class, keeps a two-way association consistent, and fixes a class that wrongly inherits from a standard container.

**Facilitation guidance:** Read DESIGN.md first: every relationship row should name the C++ technique and justify it by lifetime. Then check that two-way associations change only through one method, and that the Part 3 refactor exposes only meaningful operations and enforces the stated rule.

## Discussion & Comprehension Check Questions

---

### 1. Is a Club's list of Students composition or aggregation? Why?

*Guidance: Aggregation: students exist independently of the club, can belong to several clubs, and must not be deleted when the club is disbanded.*

### 2. Why should a two-way association be changed through one method?

*Guidance: So both sides are always updated together; updating them separately from outside lets them disagree.*

### 3. What goes wrong when SensorLog inherits from std::vector<double>?

*Guidance: Every vector operation becomes public, so callers can insert invalid readings, erase, or clear the log, bypassing the class's rules.*

## Assessment Ideas

---

- Given a paragraph describing a system, students draw text UML with correct relationship symbols and multiplicities.
- Students rewrite a small inheritance-for-reuse design using composition and delegation, and explain what was gained.

## Answer Key Notes

---

See 27-composition-aggregation/exercises/solutions/exercise1\_solution.cpp, which includes a model UML block.

## Wrap-Up & Bridge to Next Module

---

Bridge forward: "When inheritance IS the right tool, it has more to offer than Module 11 showed. Next module: inheritance modes, inheriting constructors, name hiding, final, and multiple inheritance with the diamond problem."

## Instructor Tips

---

- If time is short, protect the lifetime test (step 2) and composition over inheritance (step 5).
- Keep asking 'who owns this?' - it connects Modules 26 and 27 into one way of thinking.